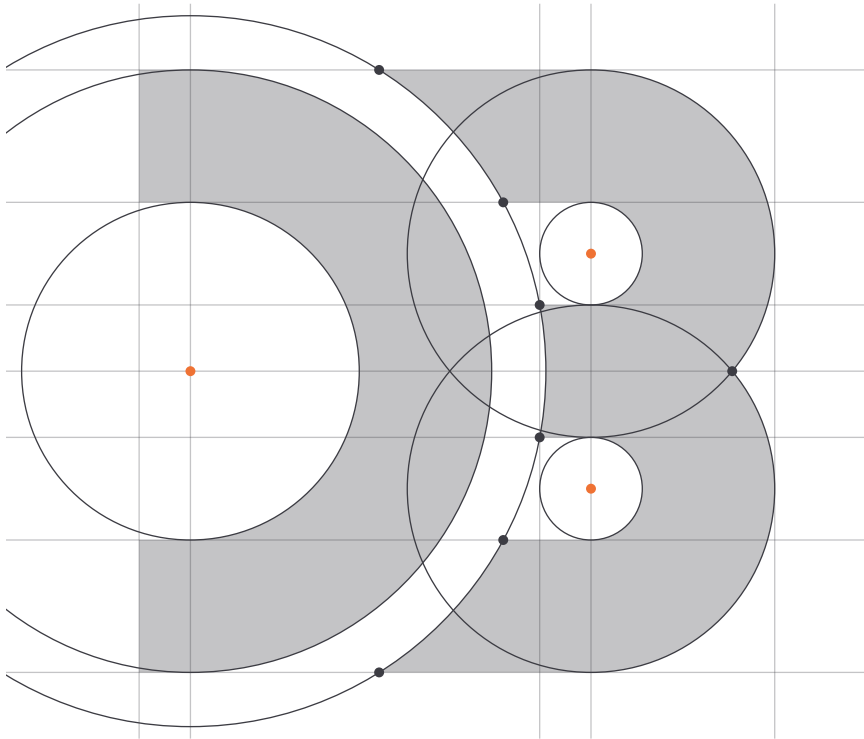


What is D3?



D3 (or **D3.js**) is a free, open-source JavaScript library for visualizing data. Its low-level approach built on web standards offers unparalleled flexibility in authoring dynamic, data-driven graphics. For more than a decade D3 has powered groundbreaking and award-winning visualizations, become a foundational building block of higher-level chart libraries, and fostered a vibrant community of data practitioners around the world.

D3 “slingshotted the field into growth, diversification and creativity that has been unprecedented” and “changed how millions of data visualizations are created across newsrooms, websites, and personal portfolios,” remarked the Information is Beautiful [2022 Test of Time Award](#). The IEEE VIS [2021 Test of Time Award](#) noted, “By creating a framework that was compelling and easy for web developers to use to author interactive visualizations, the authors have undeniably helped to bring data visualization to the mainstream. [D3] is a cornerstone contribution to this conference specifically and more generally to the success of our field as a whole.”

D3 was created by Mike Bostock in 2011. Mike co-authored the [D3 paper](#) with Jeff Heer and Vadim Ogievetsky at Stanford. Jason Davies made major contributions to D3 from 2011 to 2013, most notably to D3’s geographic projection system. Philippe Rivière has been a major contributor to D3 and its documentation since 2016. Over the years, countless kind individuals have contributed to D3 by sharing code and ideas, by teaching and answering questions, and by bringing people together to further the practice of visualization. Mike and Philippe now maintain D3 and [Observable Plot](#) at [Observable](#).

D3 is a low-level toolbox

D3 is not a charting library in the traditional sense. It has no concept of “charts”. When you

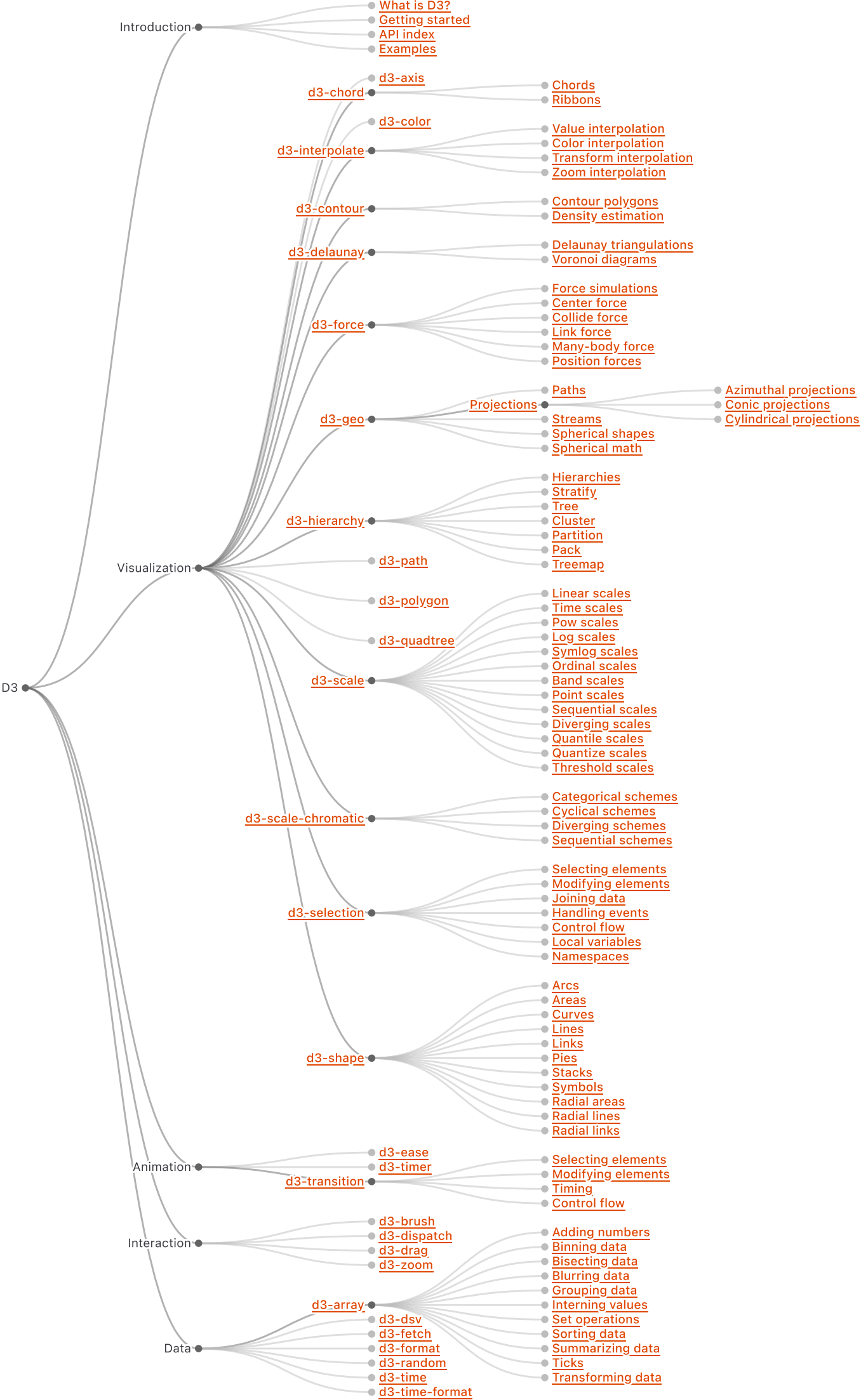
visualize data with D3, you compose a variety of primitives.

To make a [stacked area chart](#), you might use

- a [CSV parser](#) to load data,
- a [time scale](#) for horizontal position (x),
- a [linear scale](#) for vertical position (y),
- an [ordinal scale](#) and [categorical scheme](#) for color,
- a [stack layout](#) for arranging values,
- an [area shape](#) with a [linear curve](#) for generating SVG path data,
- [axes](#) for documenting the position encodings, and
- [selections](#) for creating SVG elements.

That's a lot to take in, right? But take a deep breath — you don't have to learn everything at once. Each piece can be used independently, so you can learn them individually before you fit them together. D3 is not a single monolith but rather a suite of 30 discrete libraries (or "modules"). We bundle these modules together for convenience rather than necessity so your tools are within reach as you iterate on your design.

What all's in the D3 toolbox? We recommend exploring the documentation and examples to get a sense of what's relevant to you.



TIP

Unless you need D3’s low-level control, we recommend our high-level sister library: [Observable Plot](#). Whereas a histogram in D3 might require 50 lines of code, Plot can do it in one! Plot’s concise yet expressive API lets you focus more on analyzing and visualizing data instead of web development. You can even combine Plot and D3 for the best of both.

D3 is flexible

Because D3 has no overarching “chart” abstraction, even a basic chart may require a few dozen lines of code. On the upside, all the pieces are laid out in front of you and you have complete control over what happens. You can tailor the visualization to achieve exactly what you want. D3 has no default presentation of your data — there’s just the code you write yourself. (Or copy from an example.)

Consider D3 an alternative to “doing everything yourself”, not an alternative to a high-level charting library. If you aren’t satisfied with other tools and you’re thinking of rolling your own charts using SVG or Canvas (or even WebGL), you might as well peruse D3’s toolbox! There’s almost certainly something here that will help you build the chart of your dreams without imposing on your creativity.

D3 works with the web

D3 doesn’t introduce a new graphical representation; instead, you use D3 directly with web standards such as SVG and Canvas.

The name “D3” is short for *data-driven documents*, where *documents* refers to the [Document Object Model \(DOM\)](#) standard that represents the contents of a webpage. While some of D3’s modules (such as [selections](#) and [transitions](#)) touch the DOM, others (including [scales](#) and [shapes](#)) only operate on data. D3 can also be paired with web frameworks such as React, Vue, and Svelte; see the [getting started guide](#) for recommendations.

D3’s embrace of web standards brings many benefits. For example, you can use external stylesheets to alter the appearance of charts (even in response to media queries, say for responsive charts or dark mode); you can use the debugger and element inspector to review what your code is doing; and D3’s synchronous, imperative evaluation model — calling [selection.attr](#) immediately mutates the DOM — can make it easier to debug than frameworks with complex asynchronous runtimes.

D3 is for bespoke visualization

D3 makes things possible, not necessarily easy; even simple things that should be easy are often not. To paraphrase Amanda Cox: “Use D3 if you think it’s perfectly normal to write a hundred lines of code for a bar chart.”

If you need maximal expressiveness for your bespoke visualization, you should consider D3. D3 makes sense for media organizations such as *The New York Times* or *The Pudding*, where a single graphic may be seen by a million readers, and where a team of editors can work together to advance the state of the art in visual communication.

On the other hand, D3 is overkill for throwing together a private dashboard or a one-off analysis. Don’t get seduced by whizbang examples: many of them took an immense effort to implement! If you’re constrained by time (and who isn’t?), you’d likely produce a better visualization or analysis with [Observable Plot](#).

D3 is for dynamic visualization

D3's most novel concept is its [data join](#): given a set of data and a set of DOM elements, the data join allows you to apply separate operations for *entering*, *updating*, and *exiting* elements. If you're only creating static charts (charts that don't animate or respond to user input), you may find this concept unintuitive or even bizarre because it's not needed.

The data join exists so that you can control *exactly* what happens when your data changes and update the display in response. This direct control allows extremely performant updates — you only touch the elements and attributes that need changing, without diffing the DOM — and smooth animated transitions between states. D3 shines for dynamic, interactive visualizations. (Try option-clicking the state toggles in ["512 Paths to the White House"](#) from 2012. Really.)

Next page
[Getting started](#)